Nucleic Combinators I: The Complement Involution

Working paper v0.1, Project secret3, August 11, 2026

Abstract

This paper opens research line C8, nucleic combinators, and delivers its first tier. The observation that starts the line: three of the four deep tricks of nucleic acids, inert linear text, complementary pairing, and fidelity under double copying, are available in the two-move calculus under other names, and the fourth, the polymerase, is a specifiable campaign. The delivery: the calculus's canonical duality, swap every K and S, compiled as compT with its involution law (compInvolution); a four-letter base alphabet, A as K, U as S, G as K applied to K, C as S applied to S, compiled as a datatype with its term interpretation; the pairing theorem, that the term complement of every base is the term of its Watson and Crick partner (basePairing), so that base pairing is the calculus's own light and shadow symmetry; the inertness theorem, that every base is a value of the calculus (baseVal), text that never fires; and the fidelity theorem, replicationExact, that complementing a strand twice restores it exactly, base for base. The last theorem carries the line's first slogan: the copier never copies, it complements, twice, and its round-trip exactness is structural, not proofread. One asymmetry is reported as a finding: complementation does not preserve value-hood in general, since the saturated-but-one S head maps to a K head at full valence, which fires; the alphabet therefore lives, deliberately, in the symmetric saturation zone where the two valences agree, and the valence asymmetry of the two moves, two against three, is the reason a nucleic alphabet cannot use the whole space of values. The paper closes with the specification of tier N2, the polymerase, and with a methods section that reports an infrastructure event honestly: the shared module has reached the sandbox checker's memory ceiling, this part is verified against its exact dependency slice, and modularization is registered as the program's required next infrastructure task.

1. The line and its tiers

Line C8 has four registered tiers. N1, this paper: the involution, the alphabet, pairing, inertness, fidelity. N2: the polymerase, a compiled term that walks a template strand, recruits the free complement of each base from the vessel, appends it, and recurses through the knot, with correctness stated against the strand semantics and the full bond bill metered; recruitment is the monomer consumption, the released atoms are the leaving group, and the mint law is the guarantee that every bond of the daughter is paid from the pool. N3: hybridization and melting in the vessel engine, where the existing melting-share dial is temperature and the registered prediction is that replication throughput peaks in the busy middle. N4: the codon constructor, strands read three bases at a frame as building instructions, the concrete route to the nest-builder's missing constructor. The honest boundary of the whole line, stated at the door: no helix, since the calculus has no space, which is line C7; no stacking energies, since it has no energies, which is line C6' s open half; and no mutation, since these theorems are exact, while real polymerases err and errors are where evolution lives; noise is an engine feature to be added deliberately, later, never smuggled.

2. The alphabet, and why it lives where it lives

The four bases are chosen as the four smallest values in the symmetric saturation zone: the two bare atoms, and each atom holding one copy of itself. Three theorems fix their character. basePair-

ing: the term complement of each base is the term of its partner, A to U, G to C, and back, four cases, each definitional, so Watson and Crick pairing is not modeled on top of the calculus but found inside it, as the K against S duality read letterwise. baseVal: every base is a value, hence root-inert by the progress theorem of the coordination paper, text in the exact sense of the songbook, stable the way real bases are stable. And compBaseInvolution with compInvolution: pairing is an involution at both levels, alphabet and term. The finding of the section is the boundary of the choice. Complementation does not preserve value-hood in general: the value shape S holding two, one short of the Starling's valence of three, maps under the duality to K holding two, which is the Kestrel at full valence, a live redex. The two moves' valences differ, two against three, so the duality respects inertness only where the saturation levels are matched, at zero and at one, and that is exactly where the alphabet sits. A nucleic alphabet in this calculus cannot be larger without leaving the symmetric zone, and the reason is the same asymmetry that makes K a bond sink and S a mover.

3. Fidelity by theorem

The strand is a sequence of bases; its complement is the sequence of partners; and replicationExact states that two complementations restore the strand exactly, base for base, for every strand of every length. The proof is the alphabet involution folded down the sequence, and the reading deserves its plain sentence. Real replication is semiconservative and complementary: the daughter of a strand is not its copy but its negative, and the granddaughter is the copy. In this calculus that architecture is not a design choice but a theorem of the duality: the copier never copies, it complements, twice, and the round trip is exact by structure. Where molecular biology buys fidelity with proofreading enzymes against thermal noise, the toy buys it with an equation, and the honest gloss is that this is precisely what a toy is for: it shows which part of the phenomenon is logic, so that the laboratory part, the noise and its policing, stands out as the part that is physics.

4. Toward N2: the polymerase specified

The polymerase campaign is C5-shaped and is registered with its spec. The term poly shall walk a template strand held in the songbook's inert medium, and per round: read the next base, a four-case dispatch for which the reader campaign's tag machinery is the template; recruit the free partner base from the vessel by the sharing move, which is the monomer consumption, priced by the recruitment rule; append it to the growing daughter; and recurse through the knot. Correctness target: poly applied to a template with an empty daughter reduces to the template with its full complement, and the bill, by the ledgers already compiled, is linear in the strand with the per-base cost fixed by the turnover theorems. One representation constraint is recorded from this paper's own analysis: a strand must not be encoded as a bare application chain, because composite bases at the head of a chain sit at or above valence and ignite, G being the Kestrel saturated; the inert medium is the quoted songbook, whose stillness is already a theorem, and N2' s first task is the strand encoding inside it.

Appendix A: Methods, and an infrastructure event

The shared module Proofs.idr stands at 3,726 lines after part AB, %default total, no postulates, no holes. Through part AA, at 3,644 lines, the single command idris2 —check Proofs.idr completed and is the verification, the eighth consecutive part to compile on first attempt. At part AB the checker itself was killed by the sandbox's four-gigabyte memory ceiling, twice, so the single-command verification of the whole module is not currently reproducible in this environment. Part AB is therefore verified against its exact dependency slice: a file containing, verbatim, the module's Term datatype, its Val datatype, and part AB, checked clean by the same checker; the slice ships beside the module

as SliceAB.idr. Two checker disciplines were learned in the process and are recorded for the program: the name swap collides with the prelude and is avoided as compT; and zero-argument constants are opaque to this checker's conversion, so alphabets are defined as datatypes with interpretation functions, which unfold when applied. Restoring the single command is registered on the agenda as the program's required infrastructure task, line M1: modularize the proof module into a package, one part per module under public export, with the top module's check as the one command.

Appendix B: The nucleic section (module part AB)

```
----------------------------------------------------------------------
-- Tier 2, part AB: nucleic combinators I (the complement involution)
----------------------------------------------------------------------

||| the duality of the two moves: swap every K and S
compT : Term -> Term
compT CK = CS
compT CS = CK
compT (Var x) = Var x
compT (App f a) = App (compT f) (compT a)

||| complementation is an involution: swapping twice is identity
compInvolution : (t : Term) -> compT (compT t) = t
compInvolution CK = Refl
compInvolution CS = Refl
compInvolution (Var x) = Refl
compInvolution (App f a) =
  rewrite compInvolution f in
  rewrite compInvolution a in Refl

||| the four bases, an alphabet of values at matched saturation
data Base : Type where
  BA : Base
  BU : Base
  BG : Base
  BC : Base

baseTerm : Base -> Term
baseTerm BA = CK
baseTerm BU = CS
baseTerm BG = App CK CK
baseTerm BC = App CS CS

||| Watson and Crick pairing at the alphabet level
compBase : Base -> Base
compBase BA = BU
compBase BU = BA
compBase BG = BC
compBase BC = BG

||| pairing is the duality, letter by letter: the term complement of
||| a base is the term of its paired base
basePairing : (b : Base) ->
  compT (baseTerm b) = baseTerm (compBase b)
basePairing BA = Refl
```

```
basePairing BU = Refl
basePairing BG = Refl
basePairing BC = Refl

||| every base is a value: the letters are chemically inert
baseVal : (b : Base) -> Val (baseTerm b)
baseVal BA = VK
baseVal BU = VS
baseVal BG = VK1
baseVal BC = VS1

||| pairing at the alphabet level is itself an involution
compBaseInvolution : (b : Base) -> compBase (compBase b) = b
compBaseInvolution BA = Refl
compBaseInvolution BU = Refl
compBaseInvolution BG = Refl
compBaseInvolution BC = Refl

||| a strand is a sequence of bases; its complement is the strand
||| of paired bases
Strand : Type
Strand = List Base

compStrand : Strand -> Strand
compStrand [] = []
compStrand (b :: bs) = compBase b :: compStrand bs

||| replication fidelity by theorem: complementing twice restores
||| the strand exactly, base for base
replicationExact : (s : Strand) -> compStrand (compStrand s) = s
replicationExact [] = Refl
replicationExact (b :: bs) =
  rewrite compBaseInvolution b in
  rewrite replicationExact bs in Refl
```

References

1. Watson and Crick (1953), for the pairing architecture the duality reproduces; Kornberg's polymerase work, for the payment reading of N2.
2. Project secret3, Coordination and Turnover: Valence II (v0.1), whose progress theorem certifies the bases' inertness, and The Assembly Ledger (v0.1), whose mint law is N2' s guarantee.
3. Project secret3, A Quoted Alphabet (v0.1), the inert medium for strands.
4. Kruszewski and Mikolov (2020, 2022), the neighboring vessel, for eventual replication experiments.